

Submission Date: 6th May 2026

Create a single zip file with all the files you want to submit. Name the file with the surname of all members of your group (e.g., `x_y_z.zip`). Upload your zip in the “Submission” folder of the corresponding week (Exercises → Week 2 - Transformers-Part-1 → Submission).

1 Subspace Dynamics of Multi-Head Attention

In Transformer self-attention, the input sequence is represented as

$$X \in \mathbb{R}^{n \times d_{\text{model}}}, \quad (1)$$

where n is the sequence length and d_{model} is the representation dimension.

Instead of using one single attention operation over the full representation space, multi-head attention uses h separate attention heads. For head $r \in \{1, \dots, h\}$, the projected queries, keys, and values are

$$Q_r = XW_r^Q, \quad K_r = XW_r^K, \quad V_r = XW_r^V, \quad (2)$$

with

$$W_r^Q, W_r^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_r^V \in \mathbb{R}^{d_{\text{model}} \times d_v}. \quad (3)$$

The output of one head is

$$H_r = \text{softmax} \left(\frac{Q_r K_r^\top}{\sqrt{d_k}} \right) V_r. \quad (4)$$

The outputs of all heads are concatenated and projected back to the model dimension:

$$\text{MHA}(X) = \text{Concat}(H_1, \dots, H_h) W^O, \quad (5)$$

where

$$W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}. \quad (6)$$

In the standard Transformer setting,

$$d_k = d_v = \frac{d_{\text{model}}}{h}. \quad (7)$$

Tasks

1. FLOP Analysis: Single-Head vs. Multi-Head Attention

Consider a Transformer layer with $d_{\text{model}} = 512$ and $h = 8$ heads, so that $d_k = d_v = 64$. Let n denote the sequence length.

Note: Ignore constant factors, bias terms, masking, dropout, layer normalization, feed-forward layers, and the computational cost of the softmax operation itself.

- Compute the approximate computational cost (in FLOPs) of a *single-head* attention layer using full-dimensional projections ($Q, K, V \in \mathbb{R}^{n \times 512}$). Include the cost of computing the projections, the attention score matrix $QK^\top$, and the multiplication with V .
- Compute the approximate computational cost (in FLOPs) of a *multi-head* attention layer with $h = 8$ heads ($Q_r, K_r, V_r \in \mathbb{R}^{n \times 64}$). Include the cost of all projections, attention score matrices, and value multiplications across all heads.
- Show analytically that when $hd_k = hd_v = d_{\text{model}}$, the total computational cost of multi-head attention is of the same asymptotic order as full-dimensional single-head attention.
- If the FLOP cost is comparable, justify why it is beneficial to *split* the representation into multiple heads. Comment on how this can approximate higher-rank transformations by aggregating multiple low-rank ones.

2. Conceptual Logic: Syntactic vs. Semantic Heads

Consider the two sentences:

- “The animal didn’t cross the street because it was too wide.”
- “The animal didn’t cross the street because it was too tired.”

In both sentences, the pronoun “it” is ambiguous at the surface level, and its referent changes based on context.

- In a *single-head* attention model, the attention weights for the token “it” are forced to form a single probability distribution over all tokens. Discuss why this limits the model’s ability to *simultaneously* process different types of relationships (e.g., syntax, semantics, positional proximity).
- In a *multi-head* model, suppose we define two hypothetical attention heads:
 - **Head A (Syntactic Head):** Focuses on grammatical roles (subject, object, etc.).
 - **Head B (Semantic Head):** Focuses on semantic compatibility (which noun is more plausible as the referent in context).

For each of the two sentences, describe qualitatively which token(s) “it” might attend to in Head A and in Head B. How does this illustrate the idea that multi-head attention allows the model to attend to different *aspects* of the same sequence in parallel?

2 Masked Language Modeling (MLM)

BERT is trained with a masked language modeling objective rather than a left-to-right autoregressive likelihood. Given an input sequence $\mathbf{x} = (x_1, x_2, \dots, x_T)$, a subset of token positions $\mathcal{M} \subseteq \{1, \dots, T\}$ is selected for prediction. The input sequence is corrupted to obtain $\tilde{\mathbf{x}}$, where the selected positions may be replaced by special mask tokens, random tokens, or left unchanged, depending on the corruption rule.

The model is trained to predict the original tokens at the selected positions:

$$\mathcal{L}_{\text{MLM}}(\theta) = - \sum_{i \in \mathcal{M}} \log p_{\theta}(x_i \mid \tilde{\mathbf{x}}). \quad (8)$$

This objective can be interpreted as using the following pseudo-likelihood-style factorization over the selected target tokens:

$$p_{\theta}(\mathbf{x}_{\mathcal{M}} \mid \tilde{\mathbf{x}}) \approx \prod_{i \in \mathcal{M}} p_{\theta}(x_i \mid \tilde{\mathbf{x}}). \quad (9)$$

This is not a true joint probability model over the full sequence. Instead, it is a pseudo-likelihood-style denoising objective that predicts selected tokens from a shared corrupted bidirectional context.

Tasks

1. Conditional Independence of Masked Targets

In an autoregressive language model, such as GPT, the joint probability of a sequence is decomposed exactly using the chain rule:

$$p(\mathbf{x}) = \prod_{t=1}^T p(x_t \mid x_{<t}). \quad (10)$$

In BERT’s masked language modeling objective, all selected masked tokens are predicted from the same corrupted input sequence $\tilde{\mathbf{x}}$.

- Does the MLM objective treat the masked target tokens as conditionally independent given the corrupted input $\tilde{\mathbf{x}}$? Justify your answer carefully, distinguishing between how the encoder representations are computed and how the loss factorizes.
- Consider the phrase “New York”. Suppose both tokens are selected for masking, so the corrupted input contains [MASK] [MASK]. During a single forward pass, does the model’s predicted token distribution for “New” become an input to, or condition, its prediction of “York”?

- (c) Explain why this factorization is a theoretical limitation compared to autoregressive generation. Contrast the true joint probability $p(\text{"New", "York"} \mid \text{context})$ with the product of conditionally factorized masked-token prediction terms that the MLM objective effectively trains. Why can this be problematic?

2. Span Corruption vs. Independent Token Masking

BERT typically masks individual token positions and computes the loss only on the selected masked tokens. T5's final pre-training setup uses a span corruption objective: contiguous spans of tokens are removed from the encoder input and replaced by special sentinel tokens such as $\langle X \rangle$, $\langle Y \rangle$, $\langle Z \rangle$. The decoder is then trained to reconstruct the removed spans.

For example, an original sequence such as "The animal did not cross the street because it was too wide." might be corrupted into an encoder input such as "The animal $\langle X \rangle$ the street because $\langle Y \rangle$." with a decoder target such as " $\langle X \rangle$ did not cross $\langle Y \rangle$ it was too wide $\langle Z \rangle$ ".

- (a) BERT processes the full corrupted input sequence (e.g., 512 tokens) even though the loss is computed on only $\sim 15\%$ of those tokens. Explain how T5's span corruption improves this computational inefficiency. Consider how the encoder input length and the density of the loss computation change.
- (b) Collapsing a span of length k into a single sentinel token causes tokens that were originally far apart to become adjacent in the corrupted sequence. Explain how this can negatively affect a model that uses strictly *absolute* positional embeddings, and briefly describe how T5 mitigates this issue.

3 Byte Pair Encoding (BPE)

Byte Pair Encoding (BPE) is a greedy subword tokenization algorithm. Starting from a character-level vocabulary, it repeatedly counts adjacent token pairs in the corpus and merges the most frequent pair into a new token. Because BPE makes locally optimal merge decisions, early choices can permanently affect which later tokens can or cannot be formed.

Tasks

1. Greedy Merging and Morphology

- (a) Consider the string

ababa

as the entire corpus.

Starting from the character-level tokenization $a \ b \ a \ b \ a$, list all adjacent character-pair frequencies. Then answer the following questions:

- Which two adjacent pairs are tied for highest frequency?
- If the tie is broken in favor of merging **ab**, what is the resulting tokenization after the first merge?
- If the tie is instead broken in favor of merging **ba**, what is the resulting tokenization after the first merge?
- Analyze how the choice of tie-breaking affects the final tokenization and the ability to form specific composite tokens later.

Use this example to discuss the sensitivity of greedy BPE to early merge decisions and its implications for preserving linguistically meaningful units.

- (b) The word

"unhappiness"

contains morphologically meaningful parts, such as the prefix **un-**, the root related to **happy**, and the suffix **-ness**.

Give one plausible BPE tokenization that does *not* respect these morphological boundaries. Explain why such segmentations can be problematic for morphologically rich languages.

2. Vocabulary Size, Embedding Parameters, and Rare Tokens

Let V denote the vocabulary size, d_{model} the embedding dimension, and N the total number of characters in the corpus. As V increases, tokens become longer on average, so the tokenized sequence length $L(V)$ usually decreases.

- (a) Describe the structural trade-off between embedding parameters and sequence length.

In your answer, integrate the terms $V \times d_{\text{model}}$ (for the embedding table size) and $L(V)$ (for the sequence length after tokenization). Analyze how increasing V impacts both the computational cost of the self-attention mechanism and the memory footprint of the model parameters.

- (b) At what point does increasing V yield diminishing returns for sequence compression? Explain the statistical reasons for this saturation.
- (c) Suppose V is made very large, resulting in some tokens appearing only once in the entire training corpus. Evaluate the impact of this sparsity on the quality and variance of the learned embeddings for rare tokens compared to frequent subwords.